

1: Customer Churn Prediction.

Project Overview

Title: Customer Churn Prediction using Machine Learning

Objective:

Build a machine learning model that predicts whether a telecom customer is likely to leave the company (churn) based on customer information.

Dataset:

WA_Fn-UseC_-Telco-Customer-Churn.csv

Target Variable:

Churn

- Yes = Customer leaves
- No = Customer stays

Step 1: Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Explanation

- numpy → Numerical calculations.
- pandas → Reading and manipulating datasets.
- matplotlib → Creating graphs.
- seaborn → Better-looking statistical visualizations.

Step 2: Load the Dataset

```
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
```

Step 3: Data Preprocessing

Check missing values

```
df.isnull().sum()
```

```
df.duplicated().sum()
```

```
df.drop("customerID", axis=1, inplace=True)
```

Step 4: Handle TotalCharges

Convert to numeric:

```
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
```

```
df["TotalCharges"].fillna(df["TotalCharges"].median(), inplace=True)
```

Step 5: Encode Target Variable

Convert: `df["Churn"] = df["Churn"].map({"Yes":1,"No":0})`

Now

- Yes → 1
- No → 0

Step 6: Encode Categorical Features

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
for column in df.columns:
    if df[column].dtype == 'object':
        df[column] = le.fit_transform(df[column])
```

Step 7: Feature & Target

```
X = df.drop("Churn", axis=1)
y = df["Churn"]
```

Step 8: Train-Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

80% data → Training

20% data → Testing

Step 9: Train Model

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)

model.fit(X_train, y_train)
```

Step 10: Prediction

```
y_pred = model.predict(X_test)
```

Step 11: Model Evaluation

```
accuracy_score(y_test, y_pred)

from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, y_pred)

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Expected Result

You should obtain an accuracy around **78–82%**, depending on preprocessing and the random train/test split.

Viva Questions

Q1. What is Customer Churn?

A customer stopping the use of a company's service.

Q2. Why predict churn?

To identify customers likely to leave so the company can retain them.

Q3. Which algorithm is used?

Random Forest Classifier.

Q4. Why remove customerID?

It is only an identifier and has no predictive value.

Q5. What is the target variable?

Churn

Q6. What is train-test split?

Dividing the dataset into training and testing subsets to evaluate the model.

Q7. What is accuracy?

The proportion of correct predictions among all predictions.

_____~_____

Project 2: Diabetes Prediction Using Machine Learning**Project Objective**

Build a machine learning model that predicts whether a person has diabetes based on medical attributes such as glucose level, blood pressure, BMI, insulin, age, etc.

Dataset: diabetes.csv

Target Variable: Outcome

- 0 = No Diabetes
- 1 = Diabetes

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load Dataset

```
df = pd.read_csv("diabetes.csv")
```

```
df.head()
```

```
df.info()
```

```
df.describe()
```

```
df.shape
```

Step 3: Check Missing Values

```
df.isnull().sum()
```

Step 4: Replace Invalid Zeros

```
cols = ["Glucose","BloodPressure","SkinThickness","Insulin","BMI"]
```

```
for col in cols:  
    df[col] = df[col].replace(0, df[col].median())
```

Step 5: Exploratory Data Analysis (Optional)

Correlation Heatmap

```
plt.figure(figsize=(10,8))  
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")  
plt.show()
```

Outcome Count

```
sns.countplot(x='Outcome', data=df)  
plt.show()
```

Step 6: Feature & Target

```
X = df.drop("Outcome", axis=1)  
y = df["Outcome"]
```

Step 7: Train-Test Split

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.2,  
    random_state=2  
)
```

Step 8: Feature Scaling

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

Step 9: Train Model

Use **Support Vector Machine (SVM)**

```
from sklearn.svm import SVC
```

```
model = SVC(kernel='linear')
```

```
model.fit(X_train, y_train)
```

Step 10: Prediction

```
y_pred = model.predict(X_test)
```

Step 11: Evaluate Model

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score(y_test, y_pred)
```

```
from sklearn.metrics import confusion_matrix

confusion_matrix(y_test, y_pred)

from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

Step 12: Predict for New Patient

Example:

```
input_data = (6,148,72,35,0,33.6,0.627,50)

input_array = np.asarray(input_data)
input_array = input_array.reshape(1,-1)

input_array = scaler.transform(input_array)

prediction = model.predict(input_array)

if prediction[0] == 0:
    print("The person is not diabetic.")
else:
    print("The person is diabetic.")
```

Expected Accuracy

Using the standard Pima Indians Diabetes dataset and a linear SVM, you can typically expect an accuracy of **75–80%**, depending on preprocessing and the train-test split.

Viva Questions

Q1. What is the objective of this project?

To predict whether a patient has diabetes using medical data.

Q2. What is the target variable?

Outcome

Q3. Why is feature scaling used?

SVM is sensitive to feature scales, so scaling helps improve performance.

Q4. Which algorithm is used?

Support Vector Machine (SVM).

Q5. What does Outcome = 1 mean?

The patient is diabetic.

Q6. Why replace zeros in columns like BMI and Glucose?

A value of 0 is medically unrealistic for these measurements and is treated as missing data.

Q7. Why split the dataset into training and testing sets?

To train the model on one subset of the data and evaluate its performance on unseen data.

.....Project 3: Heart Disease Prediction Using Machine Learning

Project Objective

Develop a machine learning model to predict whether a patient has heart disease based on medical parameters.

Dataset: heart_disease_data.csv

Target Variable: target

- 0 = No Heart Disease
- 1 = Heart Disease

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load Datase

```
df = pd.read_csv("heart_disease_data.csv")
```

```
df.shape
```

Step 3: Check Missing Values

```
df.isnull().sum()
```

```
df.duplicated().sum()
```

```
df.drop_duplicates(inplace=True)
```

Step 4: Exploratory Data Analysis (EDA)

Target Distribution

```
sns.countplot(x='target', data=df)
plt.title("Heart Disease Distribution")
plt.show()
```

```
plt.figure(figsize=(12,10))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

Step 5: Separate Features and Target

```
X = df.drop("target", axis=1)
y = df["target"]
```

Step 6: Train-Test Split

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=2
)
```

Step 7: Feature Scaling

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

Step 8: Train the Model

Use **Logistic Regression**.

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

Step 9: Prediction

```
y_pred = model.predict(X_test)
```

Step 10: Evaluate the Model

```
from sklearn.metrics import accuracy_score
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print(cm)
```

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))
```

Step 11: Predict for a New Patient

```
input_data = (52,1,0,125,212,0,1,168,0,1.0,2,2,3)
```

```
input_array = np.asarray(input_data)
```

```
input_array = input_array.reshape(1,-1)
```

```
input_array = scaler.transform(input_array)
```

```
prediction = model.predict(input_array)
```

```
if prediction[0] == 1:
```

```
    print("The person has heart disease.")
```

```
else:
```

```
    print("The person does not have heart disease.")
```

Expected Accuracy

With the commonly used Heart Disease dataset and Logistic Regression, the model typically achieves an accuracy of **82–88%**, depending on preprocessing and the train-test split.

Viva Questions

Q1. What is the goal of this project?

To predict whether a person has heart disease using medical features.

Q2. What is the target variable?
target

Q3. What does target = 1 indicate?
The patient has heart disease.

Q4. Why is feature scaling applied?
It standardizes feature values so the model trains more effectively.

Q5. Which algorithm is used?
Logistic Regression.

Q6. Why use Logistic Regression for this project?
It is a simple, efficient algorithm for binary classification and provides interpretable results.

Q7. What is a confusion matrix?
It summarizes the model's predictions into true positives, true negatives, false positives, and false negatives to evaluate classification performance.

_____~_____

.....**Project 4: Loan Prediction Using Machine Learning**

Project Objective

Develop a machine learning model to predict whether a loan application will be **approved or rejected** based on applicant details.

Dataset: train.csv (or loan_prediction.csv)

Target Variable: Loan_Status

- Y = Loan Approved
- N = Loan Rejected

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Load Dataset

```
df = pd.read_csv("train.csv")

df.head()

df.info()

df.describe()

df.shape
```

Step 3: Check Missing Values

```
df.isnull().sum()
```


Step 4: Handle Missing Values

Fill categorical columns using the most frequent value (mode).

```
df["Gender"].fillna(df["Gender"].mode()[0], inplace=True)
df["Married"].fillna(df["Married"].mode()[0], inplace=True)
df["Dependents"].fillna(df["Dependents"].mode()[0], inplace=True)
df["Self_Employed"].fillna(df["Self_Employed"].mode()[0], inplace=True)
df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"].mode()[0], inplace=True)
df["Credit_History"].fillna(df["Credit_History"].mode()[0], inplace=True)

df["LoanAmount"].fillna(df["LoanAmount"].median(), inplace=True)
```

Step 5: Remove Loan ID

```
df.drop("Loan_ID", axis=1, inplace=True)
```

Step 6: Encode Categorical Variables

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
```

```
for col in df.columns:
    if df[col].dtype == "object":
        df[col] = le.fit_transform(df[col])
```

Step 7: Exploratory Data Analysis (Optional)

Loan Status Distribution

```
sns.countplot(x="Loan_Status", data=df)
plt.show()

plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.show()
```

Step 8: Feature and Target

```
X = df.drop("Loan_Status", axis=1)
y = df["Loan_Status"]
```

Step 9: Train-Test Split

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Step 10: Feature Scaling

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 11: Train the Model

Use Random Forest Classifier.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42)

model.fit(X_train, y_train)
```

Step 12: Prediction

```
y_pred = model.predict(X_test)
```

Step 13: Model Evaluation

Accuracy

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix

print(confusion_matrix(y_test, y_pred))
```

Classification Report from sklearn.metrics import classification_report

```
print(classification_report(y_test, y_pred))
```

Step 14: Predict for a New Applicant

```
input_data = (1,1,0,0,6000,0,120,360,1,1)

input_array = np.asarray(input_data)
input_array = input_array.reshape(1,-1)

input_array = scaler.transform(input_array)

prediction = model.predict(input_array)

if prediction[0] == 1:
    print("Loan Approved")
else:
    print("Loan Rejected")
```

Expected Accuracy

A Random Forest model on the standard Loan Prediction dataset typically achieves an accuracy of **80–85%**, depending on preprocessing and the train-test split.

Viva Questions

Q1. What is the objective of this project?

To predict whether a loan application will be approved or rejected.

Q2. What is the target variable?

Loan_Status

Q3. Why is Loan_ID removed?

It is a unique identifier and does not contribute to predicting loan approval.

Q4. Why are missing values handled before training?

Most machine learning algorithms cannot work with missing data, so values must be imputed.

Q5. Which algorithm is used?

Random Forest Classifier.

Q6. What does Credit_History represent?

It indicates whether the applicant has a good repayment history, which is one of the most influential features for loan approval.

Q7. What evaluation metrics are used?

Accuracy, Confusion Matrix, and Classification Report.

_____~_____

Project 5: Movie Recommendation System Using Machine Learning

Project Objective

Build a recommendation system that suggests movies similar to a movie selected by the user based on movie features such as genres, keywords, cast, director, and overview.

Dataset: movies.csv

Technique Used: Content-Based Recommendation using Cosine Similarity.

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
import difflib
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

Step 2: Load Dataset

```
movies_data = pd.read_csv('movies.csv')

movies_data.head()

movies_data.shape
```

Step 3: Select Important Features

```
selected_features = [
    'genres',
    'keywords',
    'tagline',
    'cast',
    'director'
]
```

Step 4: Handle Missing Values

```
for feature in selected_features:
    movies_data[feature] = movies_data[feature].fillna('')
```

Step 5: Combine Features

```
combined_features = (
```

```

    movies_data['genres'] + ' ' +
    movies_data['keywords'] + ' ' +
    movies_data['tagline'] + ' ' +
    movies_data['cast'] + ' ' +
    movies_data['director']
)

```

Step 6: Convert Text into Numerical Features

```

vectorizer = TfidfVectorizer()

feature_vectors = vectorizer.fit_transform(combined_features)

```

Step 7: Calculate Cosine Similarity

```

similarity = cosine_similarity(feature_vectors)

```

Step 8: Take Movie Name from User

```

movie_name = input("Enter your favourite movie: ")

```

Step 9: Find Closest Matching Movie

```

list_of_all_titles = movies_data['title'].tolist()

find_close_match = difflib.get_close_matches(movie_name, list_of_all_titles)

close_match = find_close_match[0]

```

Step 10: Find Movie Index

```

index_of_movie = movies_data[movies_data.title == close_match]['index'].values[0]

```

Step 11: Get Similar Movies

```

similarity_score = list(enumerate(similarity[index_of_movie]))

sorted_similar_movies = sorted(
    similarity_score,
    key=lambda x: x[1],
    reverse=True
)

```

Step 12: Display Recommendations

```

print("Movies Suggested For You:\n")

i = 1

for movie in sorted_similar_movies:
    index = movie[0]
    title = movies_data[movies_data.index == index]['title'].values[0]

    if i < 11:
        print(i, ".", title)
        i += 1

```

Sample Output

Enter your favourite movie:
Avatar

Movies Suggested For You

1. Avatar
2. Guardians of the Galaxy
3. Star Trek

4. John Carter
5. Pacific Rim
6. Jupiter Ascending
7. Interstellar
8. Prometheus
9. The Fifth Element
10. Alien

How It Works

1. Read the movie dataset.
2. Select important features (genre, cast, director, etc.).
3. Combine all selected features into one text string.
4. Convert the text into numerical vectors using **TF-IDF**.
5. Compute similarity between every pair of movies using **Cosine Similarity**.
6. Accept a movie name from the user.
7. Find the closest matching title.
8. Recommend the top 10 most similar movies.

Expected Result

The system recommends movies that are similar to the selected movie based on their content. This is a **Content-Based Recommendation System**.

Viva Questions

Q1. What is a Recommendation System?

A system that suggests relevant items, such as movies, products, or songs, based on user preferences or item similarities.

Q2. Which type of recommendation system is used?

Content-Based Recommendation System.

Q3. What is TF-IDF?

TF-IDF (Term Frequency–Inverse Document Frequency) converts text into numerical vectors by assigning higher weights to important words.

Q4. What is Cosine Similarity?

Cosine Similarity measures how similar two vectors are by calculating the cosine of the angle between them. Values closer to 1 indicate greater similarity.

Q5. Why use `difflib.get_close_matches()`?

It helps find the closest movie title when the user enters a misspelled or slightly different name.

Q6. Why combine multiple features?

Combining genres, keywords, cast, director, and tagline provides richer information for more accurate recommendations.

Q7. What are the limitations of this system?

It only recommends movies with similar content and does not consider user ratings or viewing history.

Project 6: Spam Mail Prediction Using Machine Learning

Project Objective

Build a machine learning model to classify emails as **Spam** or **Ham (Not Spam)** using Natural Language Processing (NLP).

Dataset: mail_data.csv

Target Variable: Category

- spam = Unwanted email

- ham = Legitimate email

Algorithm: Logistic Regression

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Step 2: Load Dataset

```
mail_data = pd.read_csv("mail_data.csv")
```

```
mail_data.head()
```

```
mail_data.head()
```

```
mail_data.info()
```

```
mail_data.shape
```

Step 3: Check Missing Values

```
mail_data.isnull().sum()
```

Step 4: Handle Missing Values

Replace missing text with an empty string.

```
mail_data = mail_data.where((pd.notnull(mail_data)), "")
```

Step 5: Encode Labels

Convert:

- Spam → 1
- Ham → 0

```
mail_data.loc[mail_data['Category'] == 'spam', 'Category'] = 1
```

```
mail_data.loc[mail_data['Category'] == 'ham', 'Category'] = 0
```

Step 6: Split Features and Target

```
X = mail_data['Message']
```

```
Y = mail_data['Category']
```

Step 7: Train-Test Split

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X,
    Y,
    test_size=0.2,
    random_state=3
)
```

Step 8: Convert Text into Numerical Features

Use **TF-IDF Vectorizer**.

```
feature_extraction = TfidfVectorizer(
    min_df=1,
    stop_words='english',
```

```
        lowercase=True
    )

X_train_features = feature_extraction.fit_transform(X_train)
X_test_features = feature_extraction.transform(X_test)

Y_train = Y_train.astype('int')
Y_test = Y_test.astype('int')
```

Step 9: Train the Model

Use Logistic Regression.

```
model = LogisticRegression()

model.fit(X_train_features, Y_train)
```

Step 10: Prediction

Training prediction

```
prediction_train = model.predict(X_train_features)

prediction_test = model.predict(X_test_features)
```

Step 11: Evaluate Model

Training Accuracy

```
training_accuracy = accuracy_score(Y_train, prediction_train)

print("Training Accuracy:", training_accuracy)

testing_accuracy = accuracy_score(Y_test, prediction_test)

print("Testing Accuracy:", testing_accuracy)
```

Step 12: Predict for New Mail

```
input_mail = [
    "Congratulations! You have won a free iPhone. Click here to claim your prize."
]

input_data_features = feature_extraction.transform(input_mail)

prediction = model.predict(input_data_features)

if prediction[0] == 1:
    print("Spam Mail")
else:
    print("Ham Mail")
```

Sample Output

Training Accuracy: 0.99

Testing Accuracy: 0.97

Spam Mail

How It Works

1. Load the email dataset.
2. Replace missing values.
3. Convert spam/ham labels into numeric values.
4. Split the dataset into training and testing sets.
5. Convert email text into numerical vectors using **TF-IDF**.
6. Train a **Logistic Regression** classifier.
7. Evaluate the model.
8. Predict whether a new email is spam or not.

Expected Accuracy

With the commonly used SMS Spam Collection dataset, this approach typically achieves **96–99%** accuracy, depending on preprocessing and the train-test split.

Viva Questions

Q1. What is Spam Mail?

Spam mail is an unsolicited or unwanted email, often sent for advertising, phishing, or scams.

Q2. Which algorithm is used in this project?

Logistic Regression.

Q3. What is NLP?

Natural Language Processing (NLP) is a field of AI that enables computers to understand, process, and analyze human language.

Q4. Why is TF-IDF used?

TF-IDF converts text into numerical feature vectors by giving more importance to informative words and less importance to common words.

Q5. What does `stop_words='english'` do?

It removes common English words (such as "the", "is", "and") that usually do not help in classification.

Q6. What is the target variable?

Category

Q7. What evaluation metric is primarily used?

Accuracy Score.